

AI ZERO TO HERO | MODULE 25

System Design Foundation

From a single request to Instagram, Twitter and WhatsApp scale

46 core concepts + advanced topics + 3 real-world architectures

Sheharyar Sarmad

linkedin.com/in/sheharyar-sarmad-9b7736289

github.com/Sheharyar-Sarmad/ai-zero-to-hero

WELCOME

About this module

This is **Module 25** of my **AI Zero to Hero** repository. Up to now the journey has been about models, data and code. From here we zoom out and ask the question every real product eventually faces: *how do we build a system that keeps working when millions of people use it at once?*

I started learning system design with the free visual resource freesystemdesign.com, and this module follows its flow of topics so you can read, watch and practise side by side. Once the foundation is solid, **we will be covering advanced topics** too: consensus, distributed transactions, CQRS, probabilistic data structures, security and more. The guide ends with **project learning**: simplified architectures of Instagram, Twitter and WhatsApp.

Prerequisites

You need	Why it matters
Basic backend knowledge	Know what a server, an HTTP request, a REST endpoint, a database and an environment variable are. Having written one small API in any language is enough.
The AI Zero to Hero repo	Follow the earlier modules in Sheharyar-Sarmad/ai-zero-to-hero. System design is what turns those models and scripts into production services.
Curiosity about trade-offs	There are no perfect designs here, only choices with costs. Learning to name the trade-off is the real skill.

Where to find everything

Resource	Link
This module on GitHub	25_systemdesign_foundation
Full repository	github.com/Sheharyar-Sarmad/ai-zero-to-hero
Companion visual resource	freesystemdesign.com
Connect with me	Sheharyar Sarmad on LinkedIn

> **REMEMBER**
> System design is a conversation about **requirements, numbers and trade-offs**. Whenever you meet a new concept, ask three questions: *What problem does it solve? What does it cost? When would I not use it?*

ROADMAP

What this guide covers

Part	Theme	Concepts
1	Foundations	Web request, HTTP, APIs and JSON, latency vs throughput, stateless servers, sync vs async
2	Scaling	Load balancer, horizontal vs vertical, algorithms, health checks, auto-scaling, sessions
3	Edge and Caching	CDN, edge computing, why caches exist, cache, write-through vs write-back, invalidation, multi-tier
4	Data	SQL vs NoSQL, indexes, replication, sharding, consistent hashing, ACID, eventual consistency
5	Async and Communication	Async, message queue, pub/sub, event-driven, WebSockets, webhooks, rate limiting
6	Reliability	Single point of failure, failover, circuit breaker, retries and backoff, idempotency, disaster recovery
7	Observability and Delivery	Logging, metrics and alerting, containers, CI/CD
8	Interview and Classic Designs	How to crack the interview, URL shortener, news feed, YouTube
9	Advanced Topics	Estimation, CAP, consensus, Saga, CQRS, Bloom filters, LSM trees, ID generation, microservices, security and more
10	Project Learning	Instagram, Twitter and WhatsApp architectures

How to study this guide

- Read one chapter, then close the PDF and redraw its diagram from memory.
- Explain each concept out loud as if teaching a friend. If you get stuck, that is the gap to fill.
- Pair every chapter with the matching page on freesystemdesign.com for the animated view.
- Do the three classic designs and the three project studies on a blank page before reading the solution.

CONTENTS

Table of contents

Part 1: Foundations . . . 7

01 Anatomy of a Web Request . . . 8
02 HTTP . . . 8
03 APIs & JSON . . . 9
04 Latency vs Throughput . . . 10
05 Stateless Servers . . . 10
06 Sync vs Async . . . 11

Part 2: Scaling . . . 12

07 Load Balancer . . . 13
08 Horizontal vs Vertical Scaling . . . 13
09 Load Balancing Algorithms . . . 14
10 Health Checks . . . 14
11 Auto-scaling . . . 15
12 Session Management . . . 15

Part 3: Edge and Caching . . . 16

13 CDN . . . 17
14 Edge Computing . . . 17
15 Why Caches Exist . . . 18
16 Cache . . . 18
17 Write-Through vs Write-Back . . . 19
18 Cache Invalidation . . . 19
19 Multi-Tier Caching . . . 20

Part 4: Data . . . 21

20 SQL vs NoSQL . . . 22
21 Database Indexes . . . 22
22 Database Replication . . . 23
23 Sharding . . . 23
24 Consistent Hashing . . . 24
25 ACID Transactions . . . 24
26 Eventual Consistency . . . 25

Part 5: Async and Communication . . . 26

27 Async . . . 27
28 Message Queue . . . 27
29 Pub/Sub . . . 28
30 Event-Driven Architecture . . . 29
31 WebSockets . . . 29
32 Webhooks . . . 30
33 Rate Limiting . . . 30

Part 6: Reliability . . . 32

34 Single Point of Failure . . . 33
35 Failover . . . 33
36 Circuit Breaker . . . 34
37 Retries & Backoff . . . 34
38 Idempotency . . . 35
39 Disaster Recovery . . . 36

Part 7: Observability and Delivery . . . 37

40 Logging . . . 38
41 Metrics & Alerting . . . 38
42 Containers . . . 39
43 CI/CD . . . 39

Part 8: Interview and Classic Designs . . . 41

44 How to Crack a System Design Interview . . . 42
45 Design a URL Shortener . . . 43
46 Design a News Feed . . . 44
47 Design YouTube . . . 45

Part 9: Advanced Topics . . . 46

48 Back-of-the-Envelope Estimation . . . 47
49 CAP and PACELC . . . 47
50 Consensus: Raft and Paxos . . . 48
51 Leader Election and Distributed Locks . . . 49
52 Distributed Transactions: 2PC and Saga . . . 49
53 CQRS and Event Sourcing . . . 50
54 Bloom Filters and Probabilistic Structures . . . 51
55 B-Trees vs LSM Trees . . . 51
56 Distributed ID Generation . . . 52

57 Microservices, API Gateway and Service Mesh . . . 52
58 Kafka and Stream Processing . . . 53
59 Search Systems . . . 54
60 Geospatial Indexing . . . 54
61 Backpressure and Load Shedding . . . 55
62 Gossip, Clocks and Ordering . . . 55
63 Security Essentials . . . 56
64 Zero-Downtime Deployments . . . 57
65 System Design for AI and ML . . . 57

Part 10: Project Learning . . . 59

66 Instagram Architecture . . . 60
67 Twitter (X) Architecture . . . 61
68 WhatsApp Architecture . . . 62
69 Compare and Learn . . . 63

PART 1
Foundations

The vocabulary every other idea builds on.

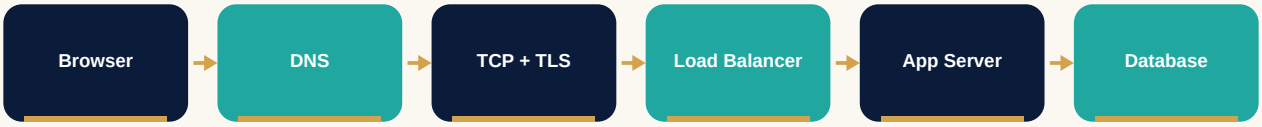
- Anatomy of a Web Request
- HTTP
- APIs and JSON
- Latency vs Throughput
- Stateless Servers
- Sync vs Async

CHAPTER 01

Anatomy of a Web Request

What really happens when you press Enter on a URL.

Typing a URL starts a surprisingly long relay race. The browser asks **DNS** to turn the name into an IP address, opens a **TCP** connection, performs a **TLS handshake** to encrypt the channel, and then sends an HTTP request. A server (often behind a load balancer) runs application code, may query a cache or database, and returns a response that the browser renders.



Key ideas

- DNS lookup is cached at the browser, OS, router and ISP levels, so repeat visits skip most of it.
- TCP and TLS add round trips before the first byte of data moves. Keep-alive and HTTP/2 reuse connections to avoid paying that twice.
- Every hop (DNS, CDN, load balancer, app, database) is a place where time is spent and things can fail.
- Static files (images, CSS, JS) should come from a CDN, not your application server.

INTERVIEW TIP
When asked to design anything, start by narrating the request path end to end. It shows you understand where latency and failure live.

CHAPTER 02

HTTP

The language clients and servers use to talk.

HTTP is a request/response protocol. A request has a **method**, a **URL**, **headers** and an optional **body**. The response carries a **status code**, headers and a body. HTTP itself is **stateless**: each request stands alone, which is exactly why it scales so well.

Method	Purpose	Safe?	Idempotent?
GET	Read a resource	Yes	Yes
POST	Create or trigger an action	No	No
PUT	Replace a resource	No	Yes
PATCH	Partially update	No	Usually not
DELETE	Remove a resource	No	Yes

Key ideas

- **2xx** success (200 OK, 201 Created, 204 No Content). **3xx** redirect. **4xx** client error (400, 401, 403, 404, 429). **5xx** server error (500, 502, 503, 504).
- HTTP/1.1 sends one request at a time per connection. HTTP/2 multiplexes many over one. HTTP/3 runs on QUIC (UDP) to avoid head-of-line blocking.

- Cache-Control, ETag and Last-Modified headers let browsers and CDNs avoid re-downloading unchanged content.
- HTTPS is simply HTTP carried inside TLS.

> **REMEMBER**
> 401 means *who are you?* (not authenticated). 403 means *I know you, but no* (not authorised). 429 means *slow down* (rate limited).

CHAPTER 03

APIs & JSON

A contract between two programs.

An **API** is the agreed way for one piece of software to ask another to do something. On the web, most APIs are **REST**: resources have URLs, HTTP verbs describe the action, and **JSON** is the data format. Others include GraphQL (client picks the fields) and gRPC (binary, fast, great between internal services).

Style	Strength	Watch out for
REST	Simple, cacheable, universal	Over-fetching or many round trips
GraphQL	One query, exactly the fields you ask for	Caching and query-cost control are harder
gRPC	Compact binary, streaming, strong typing	Not browser-native, harder to debug by eye

Key ideas

- Design nouns, not verbs: **GET /users/42/orders**, not /getUserOrders.
- Version your API (/v1/) so you can evolve without breaking clients.
- Use pagination (cursor-based scales better than offset) for any list endpoint.
- Return consistent error shapes with a machine-readable code and a human message.

> **INTERVIEW TIP**
> Mention pagination, versioning and idempotency keys when you sketch an API. It signals production experience.

CHAPTER 04

Latency vs Throughput

How fast one thing is versus how many things per second.

Latency is the time one request takes from start to finish. **Throughput** is how many requests the system completes per second. They are related but different: a highway can have high throughput (many cars) and still make each trip long. Improving one can hurt the other, for example batching raises throughput but adds waiting time to each item.

Operation (rough order of magnitude)	Time
Read from memory	100 nanoseconds
Read 1 MB sequentially from SSD	about 1 ms
Round trip inside one data centre	0.5 ms
Disk seek (spinning disk)	10 ms
Round trip across continents	100 to 150 ms

Key ideas

- Measure latency with **percentiles** (p50, p95, p99), not averages. Averages hide the slow tail that real users feel.
- Tail latency matters more at scale: if a page makes 100 calls, the slowest one decides the page speed.
- Typical levers for latency: caching, CDNs, indexes, fewer hops, connection reuse. For throughput: horizontal scaling, queues, batching.
- Bandwidth, latency and throughput are three separate ideas. Bandwidth is the pipe size.

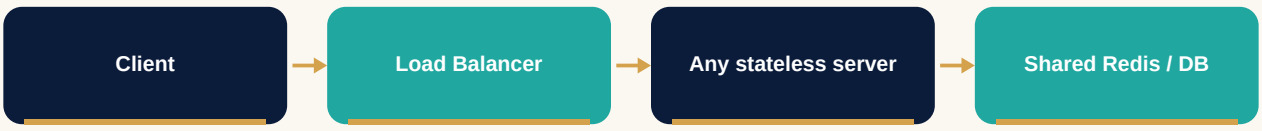
REAL WORLD
Amazon famously reported that every extra 100 ms of latency cost measurable sales. Speed is a product feature, not just an engineering metric.

CHAPTER 05

Stateless Servers

The single most important trick for scaling out.

A **stateless** server keeps no memory of past requests. Everything needed to handle a request arrives with it (a token, an ID) or is read from a shared store such as a database or Redis. Because any server can serve any request, you can add, remove or restart machines freely and put a load balancer in front.



Key ideas

- Stateful servers (data in local memory or disk) force 'sticky' routing and lose data when they die.
- Move state outward: sessions to Redis, files to object storage (S3), data to a database.
- Tokens such as JWT carry identity in the request itself, so the server need not remember the user.

- Some things are legitimately stateful (databases, WebSocket gateways). Isolate them and design them carefully.

INTERVIEW TIP
When an interviewer asks how to scale the app tier, your first sentence should be: 'Make it stateless, then add more instances.'

CHAPTER 06

Sync vs Async

Wait for the answer, or come back for it later.

In a **synchronous** call the caller waits until the work is done. In an **asynchronous** flow the caller hands the work off, gets an immediate acknowledgement and carries on. The result arrives later via polling, a callback, a webhook or a push message.

	Synchronous	Asynchronous
User experience	Instant result, but blocked if slow	Fast acknowledgement, result later
Coupling	Tight: caller needs callee up now	Loose: a queue absorbs outages
Complexity	Simple to reason about	Needs retries, ordering and status tracking
Good for	Login, payment authorisation, reads	Emails, video encoding, reports, notifications

Key ideas

- Rule of thumb: if the user needs the answer to continue, stay sync. If not, push it to a queue.
- Async turns spikes into queues instead of outages, which is a huge reliability win.

REAL WORLD
When you upload a video, the site answers 'Upload complete' at once and encodes it in the background. That is async design.

PART 2

Scaling

Serving ten users and ten million with the same idea.

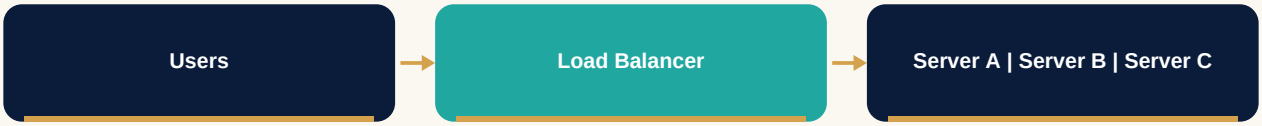
- Load Balancer
- Horizontal vs Vertical Scaling
- Load Balancing Algorithms
- Health Checks
- Auto-scaling
- Session Management

CHAPTER 07

Load Balancer

The traffic cop in front of your servers.

A **load balancer** (LB) receives incoming traffic and spreads it across a pool of servers. It improves capacity, hides server failures from users and gives you one stable address while machines come and go behind it.



Key ideas

- **Layer 4** (TCP/UDP) balancers are very fast and ignore content. **Layer 7** (HTTP) balancers can route by URL, header or cookie, and terminate TLS.
- Common choices: NGINX, HAProxy, Envoy, AWS ALB/NLB, Google Cloud LB.
- The LB itself must not become a single point of failure, so run them in pairs or use a managed, redundant service.
- Global load balancing (DNS or anycast) sends users to the nearest healthy region.

INTERVIEW TIP
Always say 'and the load balancer is redundant too.' Forgetting this is a classic single-point-of-failure slip.

CHAPTER 08

Horizontal vs Vertical Scaling

Bigger machine, or more machines?

Vertical scaling (scale up) means giving one machine more CPU, RAM or disk. **Horizontal scaling** (scale out) means adding more machines and sharing the work. Vertical is easy but hits a ceiling and keeps a single point of failure. Horizontal has no hard ceiling but needs stateless design and a way to distribute load.

	Vertical (up)	Horizontal (out)
Effort	Very low, no code change	Higher: needs LB, stateless apps
Limit	Largest machine available	Practically unlimited
Failure	Single machine = single failure	Survives losing nodes
Cost curve	Gets steeply expensive	Roughly linear with commodity hardware
Best for	Databases early on, quick fixes	Web and app tiers, caches, workers

TRADE-OFF
Start vertical because it is cheap and simple. Go horizontal when you hit limits or need high availability. Databases are the hardest layer to scale out, which leads to replication and sharding.

CHAPTER 09

Load Balancing Algorithms

How the balancer picks a server.

Different strategies suit different workloads. The right one depends on whether requests are similar in cost, whether servers are identical and whether a user needs to keep hitting the same machine.

Algorithm	How it works	Use when
Round Robin	Rotate through servers in order	Equal servers, similar requests
Weighted Round Robin	More turns for stronger servers	Mixed hardware
Least Connections	Send to server with fewest active connections	Long or uneven requests
Least Response Time	Pick the fastest, least busy server	Latency-sensitive services
IP Hash	Hash client IP to a fixed server	Need affinity without cookies
Consistent Hashing	Hash key onto a ring	Caches and sharded backends
Random (power of two choices)	Pick two at random, take the less loaded	Huge fleets, avoids herd effects

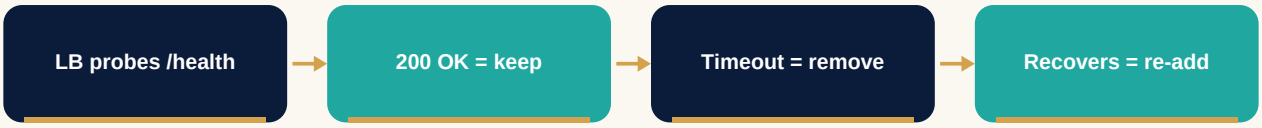
REMEMBER
Round robin is the default answer, but 'least connections' is the better answer whenever request durations vary a lot (uploads, streaming, long queries).

CHAPTER 10

Health Checks

Knowing which servers are actually alive.

A load balancer must stop sending traffic to a broken server. It does this with **health checks**: periodic probes (an HTTP GET to /health, or a TCP connect). After several failures the server is removed from rotation; after several successes it is added back.



Key ideas

- **Shallow** checks only prove the process is up. **Deep** checks verify dependencies (database, cache) but can cause cascading removals if a shared dependency blips.
- Kubernetes separates **liveness** (restart me if dead) from **readiness** (do not send traffic yet).
- Use thresholds (for example 3 failures) so a single slow response does not evict a healthy server.
- Support graceful shutdown: fail the health check first, drain in-flight requests, then stop.

CHAPTER 11

Auto-scaling

Let the system add and remove servers by itself.

Auto-scaling changes the number of running instances based on demand, so you pay for what you need and survive spikes. Scaling can be **reactive** (react to CPU, memory or queue depth), **scheduled** (every weekday 9 am) or **predictive** (machine-learned from history).

Key ideas

- Good signals: request rate per instance, queue length, p95 latency. CPU alone can mislead.
- Add a **cooldown** so the system does not flap up and down every minute.
- Scale out fast, scale in slowly. Over-provisioning for a few minutes is cheaper than an outage.
- Remember warm-up time: a new instance may need a minute to pull images and fill caches. Keep headroom or pre-warm for known events.
- Set min and max limits to guard against runaway cost or an empty fleet.

REAL WORLD
Streaming services pre-scale before a big season premiere instead of waiting for load to arrive. Predictable spikes should be scheduled, not chased.

CHAPTER 12

Session Management

Remembering who you are without remembering you on the server.

HTTP is stateless, yet users expect to stay logged in. Three common solutions exist.

Approach	How	Pros	Cons
Sticky sessions	LB pins a user to one server	Easy	Uneven load, lost on server death
Central session store	Session in Redis, cookie holds the ID	Stateless app tier, instant revocation	Extra network hop, store must be HA
Signed tokens (JWT)	Token carries claims, server verifies signature	No lookup, scales well	Hard to revoke, keep short-lived

Key ideas

- Cookies should be **HttpOnly**, **Secure** and **SameSite** to resist theft and CSRF.
- Pair short-lived access tokens with longer-lived refresh tokens.
- Always set expiry, and offer 'log out everywhere' by invalidating server-side state.

INTERVIEW TIP
Say: 'Centralise sessions in Redis so the app servers stay stateless.' It is the answer most interviewers want.

PART 3

Edge and Caching

The fastest request is the one that never reaches your server.

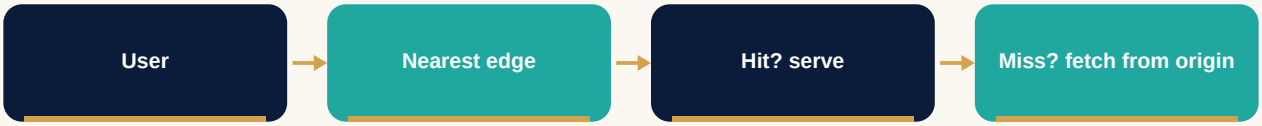
- CDN
- Edge Computing
- Why Caches Exist
- Cache
- Write-Through vs Write-Back
- Cache Invalidation
- Multi-Tier Caching

CHAPTER 13

CDN

Put content physically close to the user.

A **Content Delivery Network** is a worldwide fleet of servers (points of presence) that cache your static files near users. A viewer in Lahore gets an image from a nearby edge instead of a data centre across the planet, cutting latency and shielding your origin from load.



Key ideas

- **Pull CDN**: edge fetches from origin on first request. **Push CDN**: you upload content ahead of time.
- Control freshness with Cache-Control max-age and by versioning file names (app.3f9a.js).
- CDNs also absorb DDoS traffic, terminate TLS and compress responses.
- Great for images, video segments, JS and CSS, downloads and even cacheable API responses.
- Providers: Cloudflare, Akamai, Fastly, CloudFront.

INTERVIEW TIP
For any media-heavy design (YouTube, Instagram), the CDN is not optional. State it early and explain how it protects the origin.

CHAPTER 14

Edge Computing

Run code near the user, not only files.

Edge computing moves computation, not just caching, to locations close to users. Edge functions (Cloudflare Workers, Lambda@Edge, Vercel Edge) can personalise pages, check auth, rewrite URLs, run A/B tests or block abusive traffic in a few milliseconds without a round trip to a central region.

Key ideas

- Best for lightweight, latency-critical logic: auth checks, redirects, geo routing, header manipulation, bot filtering.
- Limits: short execution time, small memory, and data may live far away. A chatty edge function calling a distant database loses its advantage.
- Pairs with globally replicated or edge-friendly data stores for read-heavy use cases.
- IoT and AI are driving edge use: process locally, send only summaries upstream.

TRADE-OFF
Edge buys speed but fragments your logic across many locations and makes consistent data harder. Use it for the thin front layer, keep the core central.

CHAPTER 15

Why Caches Exist

Because memory is fast and recomputing is expensive.

Reading from a database or calling another service is slow compared with reading memory. Most traffic is also repetitive: a small number of items (popular posts, product pages) receive most of the requests. A **cache** stores the results of expensive work so the next request is almost free.

Key ideas

- Caching trades **freshness for speed** and **memory for compute**.
- Works because of **locality**: temporal (recent items get reused) and the 80/20 skew of popularity.
- Reduces load on databases, cuts cost and smooths traffic spikes.
- Measure the **hit ratio**. A cache with a 50% hit rate may not be worth its complexity; 95%+ is transformative.

> **REMEMBER**
> Do not cache first. Cache what you have measured to be slow and repeated.

CHAPTER 16

Cache

Patterns for reading through a cache.

The most common pattern is **cache-aside** (lazy loading): the app checks the cache first; on a miss it reads the database, stores the value in the cache and returns it. Redis and Memcached are the usual stores.



Eviction policy	Idea
LRU	Drop the least recently used item (most popular default)
LFU	Drop the least frequently used item
FIFO	Drop the oldest item
TTL	Items expire after a set time regardless of use

Key ideas

- Cache stampede (thundering herd): when a hot key expires, thousands of requests hit the database together. Fix with locking, request coalescing, jittered TTLs or early refresh.
- Cache penetration: requests for keys that never exist bypass the cache. Fix by caching 'not found' briefly or using a Bloom filter.
- Keys should be specific and versioned; values should be compact.

> **INTERVIEW TIP**
> Mention LRU eviction and a TTL when you add a cache. Then name stampede protection to stand out.

CHAPTER 17

Write-Through vs Write-Back

What happens to the cache when data changes.

Writes need a strategy so the cache and the database do not drift apart.

Strategy	Write path	Pros	Cons
Write-through	Write to cache and DB together, ack after both	Cache always fresh, simple durability	Slower writes
Write-back (behind)	Write to cache, ack, flush to DB later	Very fast writes, batching	Data loss if cache dies before flush
Write-around	Write straight to DB, skip cache	Avoids polluting cache with rarely read data	First read is a miss
Cache-aside + invalidate	Update DB, delete cache key	Most common and safe	Brief window of staleness

TRADE-OFF
Write-back is the speed champion and the durability risk. Use it for things like counters and analytics where losing a few seconds of data is acceptable, never for payments.

CHAPTER 18

Cache Invalidation

One of the two hard problems in computer science.

Cached data goes stale when the source changes. **Invalidation** is deciding when to remove or refresh it. Get it wrong and users see old prices, old profiles or deleted content.

Key ideas

- **TTL expiry**: simplest, accepts bounded staleness.
- **Explicit invalidation**: delete or update the key when the source changes. Precise but easy to miss a code path.
- **Event-driven**: a change stream (CDC or message) triggers cache updates across services.
- **Versioned keys**: put a version in the key (user:42:v7) so new data simply uses a new key.
- **Stale-while-revalidate**: serve the old value instantly and refresh in the background.
- Race condition to know: a slow reader can write an old value into the cache after a newer update. Short TTLs or versioning reduce the damage.

REAL WORLD
CDNs use the same idea with purge APIs and hashed file names, so a new deploy never serves yesterday's JavaScript.

CHAPTER 19

Multi-Tier Caching

Layers of caches, each catching what the last one missed.

Real systems stack caches from the user outward: browser cache, CDN edge, API gateway or reverse proxy, application-local cache, distributed cache (Redis) and finally the database's own buffer pool. Each layer is bigger and slower than the one before.



Key ideas

- The closer to the user a hit happens, the cheaper and faster it is.
- In-process caches are the fastest but each server holds its own copy, so they can disagree. Keep them tiny with short TTLs.
- Define ownership of each layer's TTL and invalidation to avoid mysterious stale data.
- Every extra layer multiplies debugging difficulty. Add only the tiers your numbers justify.

INTERVIEW TIP

Draw the layers on the board and annotate approximate hit rates. It shows you think in numbers.

PART 4

Data

Where truth lives and how it survives growth.

- SQL vs NoSQL
- Database Indexes
- Database Replication
- Sharding
- Consistent Hashing
- ACID Transactions
- Eventual Consistency

CHAPTER 20

SQL vs NoSQL

Choosing the right shape for your data.

SQL (relational) databases store rows in tables with a fixed schema, support joins and strong transactions. **NoSQL** is a family of stores that relax some of those guarantees for flexibility or scale: key-value, document, wide-column and graph.

Type	Examples	Best for
Relational	PostgreSQL, MySQL	Structured data, joins, transactions, finance
Key-value	Redis, DynamoDB	Sessions, caches, lookups by key at huge scale
Document	MongoDB, Firestore	Flexible, nested records such as profiles or catalogues
Wide-column	Cassandra, HBase	Massive write throughput, time-series, feeds
Graph	Neo4j	Relationships: friends, recommendations, fraud rings

INTERVIEW TIP
Do not say 'NoSQL scales, SQL does not.' Say: 'I start with PostgreSQL for correctness and add specialised stores where the access pattern demands it.' Modern SQL scales much further than people assume.

CHAPTER 21

Database Indexes

The book index that saves you from reading every page.

Without an **index**, a query scans every row (O(n)). An index is a separate sorted structure, usually a **B-tree**, that lets the database jump straight to matching rows in O(log n). It is the single cheapest performance win in most systems.

Key ideas

- Index columns used in WHERE, JOIN and ORDER BY clauses.
- **Composite indexes** follow the leftmost-prefix rule: an index on (country, city) helps country and country+city, not city alone.
- Every index slows writes and uses space, because each insert or update must maintain it.
- Covering indexes hold all columns a query needs, so the table is never touched.
- Low-cardinality columns (for example a boolean) rarely benefit from an index.
- Use EXPLAIN to see whether a query uses an index or does a full scan.

TRADE-OFF
Indexes speed reads and tax writes. A write-heavy table with ten indexes may be slower than the problem it solved.

CHAPTER 22

Database Replication

Copies of your data for speed and survival.

Replication keeps copies of the database on several machines. The common setup is **leader-follower**: all writes go to the leader, which streams changes to followers that serve reads. This scales read traffic and gives you a standby if the leader dies.



Key ideas

- **Synchronous** replication waits for followers (safe, slower). **Asynchronous** does not (fast, may lose recent writes on failover).
- **Replication lag** means a follower may be a little behind. A user who just posted and then reads from a lagging replica may not see their own post. Fix with read-your-writes routing.
- **Multi-leader** and **leaderless** (Dynamo-style) setups allow writes in several places but require conflict resolution.
- Replication is for availability and read scale. It does not help with write scale or data size, which is where sharding comes in.

> **REAL WORLD**
> Most web apps are 90%+ reads, so a leader plus several read replicas carries a surprising amount of traffic.

CHAPTER 23

Sharding

Splitting data across machines when one cannot hold it.

Sharding (horizontal partitioning) splits a large dataset into pieces called shards, each stored on a different server. Every record belongs to exactly one shard, chosen by a **shard key**. This scales both storage and write throughput.

Strategy	How	Risk
Range	Key ranges (A-F, G-M ...)	Hot spots if keys are skewed or sequential
Hash	hash(key) mod N	Even spread, but resharding moves lots of data
Directory	Lookup table maps key to shard	Lookup service becomes critical
Geo	Shard by region	Uneven population, cross-region queries

Key ideas

- Choosing the shard key is the most important decision. It should spread load evenly and keep related data together.
- Cross-shard joins and transactions are expensive. Design access patterns to stay inside one shard.
- **Hot shards** (a celebrity account) need special handling, such as splitting the key or caching.

- Resharding is painful. Plan for it, or use consistent hashing to soften it.

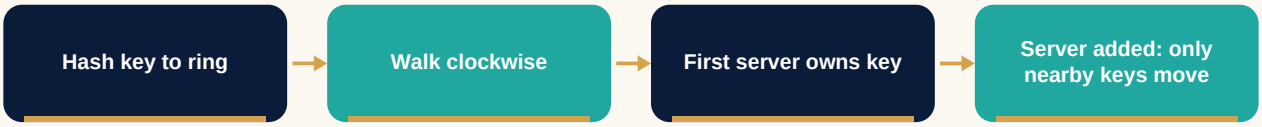
INTERVIEW TIP
Shard last. Try indexes, caching and replication first, then shard and name the shard key out loud with its trade-offs.

CHAPTER 24

Consistent Hashing

Add or remove servers without reshuffling everything.

With simple hashing (hash mod N), changing N remaps almost every key, which would flush a whole cache cluster at once. **Consistent hashing** places servers and keys on a circular ring. A key belongs to the first server clockwise from it, so adding or removing a server moves only about 1/N of the keys.



Key ideas

- **Virtual nodes**: each physical server appears many times on the ring to smooth out uneven distribution and let stronger machines take more.
- Replication on the ring: store each key on the next few servers clockwise for fault tolerance.
- Used by Cassandra, DynamoDB, Memcached clients, CDNs and many load balancers.
- Related ideas: rendezvous hashing and jump hash, which solve the same problem differently.

REMEMBER
Mod-N hashing: add one server, lose nearly all cache hits. Consistent hashing: add one server, lose about 1/N.

CHAPTER 25

ACID Transactions

The four promises of a reliable database.

A **transaction** groups several operations so they succeed or fail together. The classic example is moving money: debit one account and credit another, never only one.

Letter	Meaning	Plain English
A	Atomicity	All steps happen or none do
C	Consistency	Rules and constraints always hold after the transaction
I	Isolation	Concurrent transactions do not see each other's half-done work
D	Durability	Once committed, data survives crashes

Key ideas

- Isolation levels trade safety for speed: read committed, repeatable read, snapshot, serializable. Weaker levels allow anomalies such as dirty, non-repeatable and phantom reads or write skew.
- Databases implement atomicity and durability with a write-ahead log (WAL), and isolation with locks or MVCC (multi-version concurrency control).
- Distributed ACID across shards needs protocols like two-phase commit, which are slow and fragile (see Advanced Topics).

INTERVIEW TIP
Anything involving money, inventory or bookings deserves ACID. Say so explicitly when choosing the database.

CHAPTER 26

Eventual Consistency

Everyone will agree, just not instantly.

In a distributed system with replicas, updates take time to travel. **Eventual consistency** promises that if writes stop, all replicas will converge to the same value. In exchange you get high availability and low latency, even across regions.

Model	Guarantee
Strong consistency	Every read sees the latest write (feels like one copy)
Read-your-writes	You always see your own updates
Monotonic reads	You never see time go backwards
Causal	Effects are never seen before their causes
Eventual	Replicas converge if updates stop

Key ideas

- Fine for likes, view counts, feeds and recommendations. Not fine for bank balances or seat reservations.
- Conflict resolution tools: last-write-wins (simple, can lose data), vector clocks, and CRDTs (data types designed to merge automatically).
- Anti-entropy mechanisms such as read repair, hinted handoff and Merkle trees keep replicas converging.

TRADE-OFF
This is the CAP theorem in practice: during a network partition you choose between consistency and availability. Pick per feature, not per company.

PART 5

Async and Communication

Letting components talk without waiting on each other.

- Async
- Message Queue
- Pub/Sub
- Event-Driven Architecture
- WebSockets
- Webhooks
- Rate Limiting

CHAPTER 27

Async

Decoupling work from the request that caused it.

Asynchronous processing separates *accepting* work from *doing* work. A web request enqueues a job and returns quickly; background **workers** process it at their own pace. This protects user-facing latency and lets you scale workers independently.



Key ideas

- Return **202 Accepted** with a job ID, and let the client poll a status endpoint or receive a push.
- Workers must be **idempotent** because jobs can be delivered more than once.
- Use a dead-letter queue for jobs that keep failing so they do not block the rest.
- Typical jobs: email, image resizing, video encoding, PDF reports, search indexing, ML inference batches.

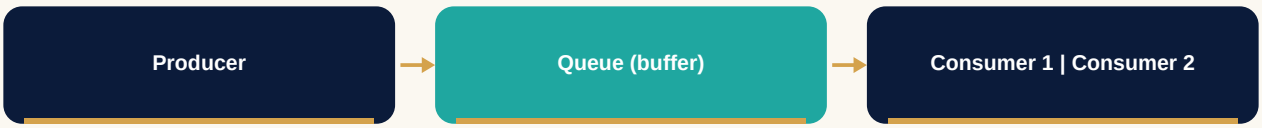
> **REAL WORLD**
> Many AI products work this way: you submit a long generation or training request, receive a job ID and get results when the GPU worker finishes.

CHAPTER 28

Message Queue

A durable line of work between producer and consumer.

A **message queue** (RabbitMQ, Amazon SQS, Redis streams) stores messages until a consumer processes them. Producers and consumers never talk directly, so either side can slow down, fail or scale without breaking the other.



Key ideas

- **Point-to-point**: each message goes to one consumer, which gives you load sharing.
- Delivery guarantees: **at-most-once** (may lose), **at-least-once** (may duplicate; most common) and **exactly-once** (hard, usually emulated with idempotent consumers).
- Consumers acknowledge a message only after finishing; otherwise it is redelivered after a visibility timeout.
- Queues provide **backpressure** and absorb bursts. Watch queue depth as a key health metric.

- Kafka is a distributed **log** rather than a classic queue: messages are retained and can be replayed by many consumer groups.

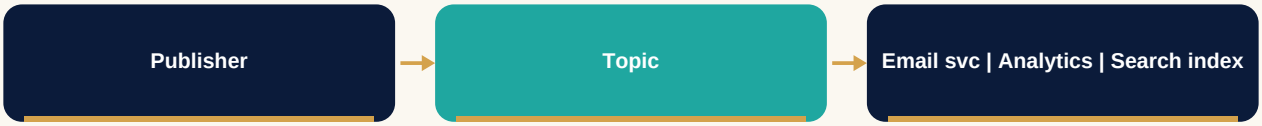
> **INTERVIEW TIP**
> Say 'at-least-once delivery plus idempotent consumers' and you will sound like you have run this in production.

CHAPTER 29

Pub/Sub

One event, many interested listeners.

In **publish/subscribe**, a publisher sends a message to a **topic** without knowing who is listening. Every subscriber to that topic gets its own copy. Adding a new consumer means subscribing, with no change to the publisher.



	Queue	Pub/Sub
Delivery	One consumer gets each message	Every subscriber gets a copy
Use for	Distributing work	Broadcasting facts
Example	Resize this image	Order was placed

Key ideas

- Tools: Kafka topics, Google Pub/Sub, AWS SNS, Redis Pub/Sub, NATS.
- Combine them: SNS fans out to several SQS queues, each with its own workers (the fan-out pattern).
- Keep message schemas versioned and backward compatible.

> **REAL WORLD**
> When you place an order, one event can trigger payment, inventory, email, analytics and fraud checks, all independently.

CHAPTER 30

Event-Driven Architecture

Systems that react to things that happened.

In an **event-driven** design, services communicate by emitting **events** (past-tense facts such as *OrderPlaced*) to a broker, and other services react. This gives loose coupling, easy extension and natural audit trails.

Key ideas

- **Event notification**: small event, listeners fetch details. **Event-carried state transfer**: event holds all data. **Event sourcing**: events are the source of truth.
- Benefits: independent scaling, resilience to downstream outages, and easy addition of new features.
- Costs: harder to trace a business flow, eventual consistency, duplicate and out-of-order events, schema evolution.
- Use correlation IDs and distributed tracing so you can follow one flow across services.
- **Choreography** (services react on their own) is flexible; **orchestration** (a coordinator directs steps) is easier to understand and monitor.

TRADE-OFF
Event-driven systems are easy to extend and hard to debug. Invest in tracing from day one.

CHAPTER 31

WebSockets

A two-way, always-open line between client and server.

HTTP is request/response: the server cannot speak first. A **WebSocket** upgrades an HTTP connection into a persistent, full-duplex channel so either side can send messages at any time with low overhead. Perfect for chat, live scores, collaborative editing, games and trading dashboards.

Technique	How	Best for
Short polling	Client asks every few seconds	Simple, low-frequency updates
Long polling	Server holds request until data exists	Fallback when WebSockets unavailable
Server-Sent Events	One-way server to client stream	Notifications, live feeds
WebSocket	Two-way persistent channel	Chat, games, collaboration

Key ideas

- Connections are **stateful**, so scaling needs care: each gateway server holds many thousands of sockets.
- To message a user on another server, use a pub/sub backbone (Redis, Kafka) or a presence registry mapping user to gateway.
- Handle reconnects, heartbeats (ping/pong) and message replay using sequence numbers.

- Load balancers must support long-lived connections and sensible idle timeouts.

INTERVIEW TIP
For chat, say: 'WebSocket gateway tier, a presence service, and a pub/sub layer to route between gateways.'

CHAPTER 32

Webhooks

Server-to-server notifications: 'call me when it happens'.

A **webhook** is an HTTP callback. Instead of you polling an external service ('is the payment done yet?'), you register a URL and the service sends you a POST when the event occurs. Stripe, GitHub and Twilio all work this way.



Key ideas

- **Verify the signature** (HMAC) so attackers cannot forge events.
- Respond quickly with 200 and process in the background; providers retry on timeouts.
- Expect **duplicates and out-of-order** deliveries. Use the event ID for idempotency.
- Providers retry with exponential backoff and may disable endpoints that keep failing. Monitor your receiver.
- If you build webhooks for others, offer retries, a delivery log and a replay button.

REMEMBER
Webhooks are push over HTTP between servers. WebSockets are a live channel between a client and a server. They are not interchangeable.

CHAPTER 33

Rate Limiting

Protecting your system from too much of a good thing.

A **rate limiter** caps how many requests a client may make in a time window. It stops abuse, brute-force attacks and noisy neighbours, keeps costs predictable and protects downstream systems. Exceeding the limit returns **429 Too Many Requests** with a Retry-After header.

Algorithm	Idea	Notes
Fixed window	Count per minute, reset on the minute	Simple; allows double bursts at boundaries
Sliding window log	Store timestamps of requests	Accurate, memory heavy
Sliding window counter	Blend current and previous windows	Good accuracy and cheap
Token bucket	Tokens refill at a rate; each request spends one	Allows bursts; very popular

Algorithm	Idea	Notes
Leaky bucket	Requests drain at a constant rate	Smooths traffic

Key ideas

- Limit by user ID, API key, IP, or endpoint, and use different tiers for free and paid plans.
- In a cluster, keep counters in Redis with atomic operations (INCR with expiry, or a Lua script).
- Place limiters at the API gateway or edge to reject early and cheaply.
- Decide fail-open vs fail-closed if the limiter store is down.

INTERVIEW TIP
Token bucket is the safest default answer. Mention Redis for distributed counters and 429 with Retry-After.

PART 6

Reliability

Assume everything fails. Design so users never notice.

- Single Point of Failure
- Failover
- Circuit Breaker
- Retries and Backoff
- Idempotency
- Disaster Recovery

CHAPTER 34

Single Point of Failure

The one component whose death takes everything down.

A **single point of failure** (SPOF) is any component with no backup. Hunt for it by asking of every box in your diagram: *if this disappeared right now, what happens?* The cure is **redundancy** at every layer.

Layer	Typical SPOF	Remedy
DNS	One DNS provider	Multiple providers
Load balancer	One LB	Active-passive or managed multi-AZ LB
App tier	One server	Several stateless instances across zones
Database	One primary	Replicas with automatic failover
Network and power	One zone or region	Multi-AZ, multi-region
People	One engineer who knows everything	Runbooks, on-call rotation

Key ideas

- Availability is measured in nines: 99.9% allows about 8.8 hours of downtime a year; 99.99% about 52 minutes; 99.999% about 5 minutes.
- Redundancy only helps if failures are independent. Two servers on one power strip are not redundant.

> **INTERVIEW TIP**
> After drawing your design, walk through it box by box naming each SPOF and its fix. Interviewers love that.

CHAPTER 35

Failover

Automatically switching to the backup.

Failover is the process of moving traffic from a failed component to a standby. It needs reliable **failure detection** (health checks, heartbeats) and a clear switch mechanism.

Mode	Description	Trade-off
Active-passive	Standby idle until the primary dies	Simple, but standby capacity is wasted and warm-up takes time
Active-active	All nodes serve traffic, survivors absorb load	Efficient, but needs spare headroom and careful data sync

Key ideas

- **Split brain**: two nodes both believe they are primary and accept writes. Prevent with quorum, leases or fencing tokens.
- Define **RTO** (how long you can be down) and **RPO** (how much data you can lose) to choose the right level of investment.

- Test failover regularly. An untested failover plan is a hope, not a plan. Chaos engineering (Netflix Chaos Monkey) makes this routine.

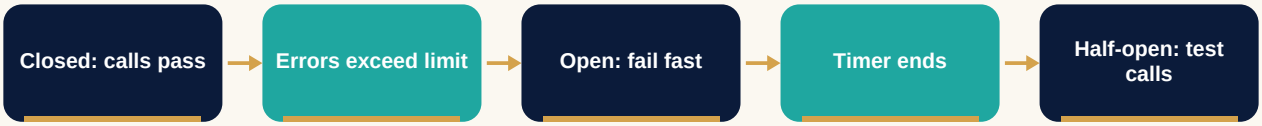
REAL WORLD
Managed databases such as RDS and Cloud SQL promote a replica automatically in about a minute. Your app must reconnect gracefully.

CHAPTER 36

Circuit Breaker

Stop calling a service that is clearly broken.

When a dependency is failing or slow, continuing to call it wastes threads, piles up timeouts and can drag your whole system down in a **cascading failure**. A **circuit breaker** watches the error rate and, past a threshold, *opens* to fail fast instead of calling.



Key ideas

- **Closed** is normal. **Open** rejects immediately and returns a fallback. **Half-open** lets a few trial requests through; success closes it, failure re-opens it.
- Pair with **timeouts** on every network call and **bulkheads** (separate thread or connection pools per dependency) so one slow service cannot starve the rest.
- Good fallbacks: cached data, a default value, a degraded feature, or a friendly error.
- Libraries: Resilience4j, Polly, Hystrix (retired), Envoy and service-mesh outlier detection.

INTERVIEW TIP
Mention circuit breaker, timeout and fallback together: it is the standard resilience trio.

CHAPTER 37

Retries & Backoff

Try again, but wisely.

Many failures are transient: a blip, a brief overload, a failover. A retry often succeeds. But naive retries make outages worse, because thousands of clients hammer a recovering server at the same moment.



Key ideas

- **Exponential backoff**: wait 1s, 2s, 4s, 8s... between attempts.
- **Jitter**: add randomness so clients do not retry in lockstep (the thundering herd).
- Cap both the number of attempts and the total time. Retrying forever is a bug.

- Retry only **retryable** errors (timeouts, 503, 429). Never retry a 400 or 404.
- Retry only **idempotent** operations, or attach an idempotency key.
- Beware of **retry amplification**: three layers each retrying three times means 27 calls for one request. Retry at one layer and use retry budgets.

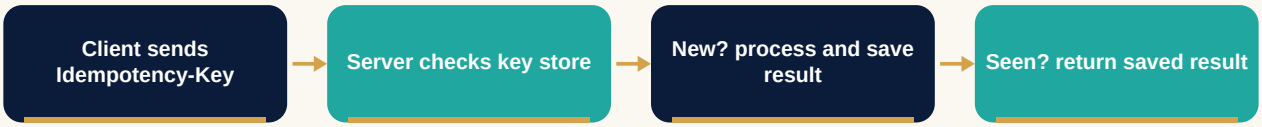
REMEMBER
Backoff + jitter + limits + idempotency. If you can recite those four, you understand retries.

CHAPTER 38

Idempotency

Doing it twice has the same effect as doing it once.

An operation is **idempotent** if repeating it leaves the system in the same state as running it once. Setting a value to 5 is idempotent; adding 5 is not. Because networks time out and clients retry, any important write must survive being delivered twice. Think of a payment charged twice because the user pressed Pay again.



Key ideas

- Clients generate a unique **idempotency key** (UUID) per logical operation and send it in a header. Stripe popularised this.
- Server stores the key with the result for a window (for example 24 hours) and returns the stored result on repeats.
- Use a unique constraint in the database so a duplicate insert simply fails.
- Natural idempotency: PUT, DELETE and 'set status = shipped' are idempotent by design. Prefer them when you can.
- Message consumers should deduplicate by message ID because queues are at-least-once.

INTERVIEW TIP
Whenever money or orders are involved, bring up idempotency keys unprompted.

CHAPTER 39

Disaster Recovery

Surviving the loss of an entire data centre or region.

Disaster recovery (DR) is the plan for recovering from major events: region outages, data corruption, ransomware, human error. Two numbers drive it. **RTO** (Recovery Time Objective) is how fast you must be back. **RPO** (Recovery Point Objective) is how much recent data you can afford to lose.

Strategy	RTO / RPO	Cost	Description
Backup and restore	Hours / hours	$	Periodic backups restored into a fresh environment
Pilot light	Tens of minutes	$$	Core data replicated; servers off until needed
Warm standby	Minutes	$$$	Scaled-down copy running continuously
Multi-site active-active	Near zero	$$$$	Full capacity in several regions at once

Key ideas

- Follow the **3-2-1 rule**: 3 copies, 2 media types, 1 off-site. Make at least one copy immutable.
- Backups you have never restored are not backups. Rehearse restores and game days.
- Replication copies mistakes instantly (a bad DELETE replicates too), so keep point-in-time recovery as well.
- Document runbooks and keep them where they are reachable when your main systems are down.

TRADE-OFF
Each step toward zero RTO and RPO multiplies cost. Ask the business what an hour of downtime actually costs before choosing.

PART 7

Observability and Delivery

You cannot fix what you cannot see, and cannot ship what you cannot deploy.

- Logging
- Metrics and Alerting
- Containers
- CI/CD

CHAPTER 40

Logging

The diary of your system.

Logs are timestamped records of events. They answer *what exactly happened?* after something goes wrong. In distributed systems the logs of hundreds of instances must be shipped to one searchable place.



Key ideas

- Use **structured logging** (JSON with fields) instead of free text so you can filter by user, request or error code.
- Add a **correlation (trace) ID** to every log line of a request and pass it between services.
- Use levels sensibly: DEBUG, INFO, WARN, ERROR. Do not log secrets, passwords or personal data.
- Logs are expensive at scale. Sample noisy ones, set retention and keep hot and cold tiers.
- Stacks: ELK/OpenSearch, Grafana Loki, Datadog, Splunk, CloudWatch.

REAL WORLD
The first debugging question in a microservice system is 'give me the trace ID.' Without it you are searching blind.

CHAPTER 41

Metrics & Alerting

Numbers that tell you if you are healthy.

Metrics are numeric measurements over time (requests per second, error rate, CPU, queue depth). They are cheap to store and perfect for dashboards and alarms. **Alerting** pages a human when something needs action.

Framework	Signals
Four Golden Signals (Google SRE)	Latency, Traffic, Errors, Saturation
RED (services)	Rate, Errors, Duration
USE (resources)	Utilisation, Saturation, Errors

Key ideas

- The three pillars of observability: **logs, metrics and traces**. Metrics tell you something is wrong, traces show where, logs explain why.
- Define **SLIs** (what you measure), **SLOs** (the target, such as 99.9% of requests under 300 ms) and **SLAs** (the contract with penalties).
- An **error budget** (100% minus the SLO) lets teams trade reliability against shipping speed.
- Alert on **symptoms users feel** (error rate, latency), not every cause (CPU 81%). Every alert should be actionable, or delete it.

- Tools: Prometheus and Grafana, Datadog, New Relic, OpenTelemetry for traces.

INTERVIEW TIP
End any design with 'and here is how I would monitor it': name the golden signals and one or two SLOs.

CHAPTER 42

Containers

Ship the app together with everything it needs.

A **container** packages code, runtime, libraries and settings into a lightweight, isolated unit that runs the same on a laptop, in CI and in production. Unlike virtual machines, containers share the host operating system kernel, so they start in seconds and use far fewer resources.

	Virtual machine	Container
Isolation	Full OS per VM, strong	Process-level, shares kernel
Size	Gigabytes	Megabytes
Start-up	Minutes	Seconds or less
Density	Tens per host	Hundreds per host

Key ideas

- Docker builds **images** (blueprints) from a Dockerfile; running instances are **containers**.
- **Kubernetes** orchestrates containers: scheduling, scaling, self-healing, rolling updates, service discovery and config.
- Containers should be **stateless and disposable**. Store data in volumes or external services.
- Keep images small, pin versions, run as non-root and scan for vulnerabilities.

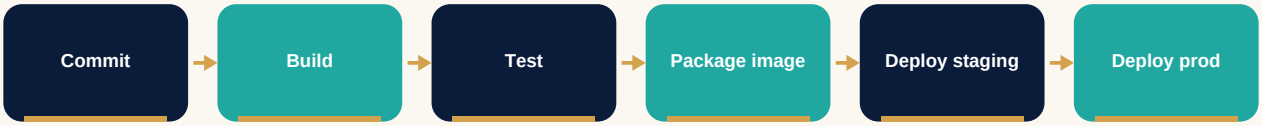
REAL WORLD
Containers are why 'it works on my machine' stopped being an excuse. They are also the natural unit for packaging ML model servers.

CHAPTER 43

CI/CD

From commit to production, automatically and safely.

Continuous Integration merges code often and runs automated builds and tests on every change. **Continuous Delivery** keeps the main branch always releasable; **Continuous Deployment** goes further and releases every passing change to production automatically.



Release strategy	How	Benefit
Rolling	Replace instances gradually	No extra capacity needed

Release strategy	How	Benefit
Blue-green	Two environments, flip traffic	Instant rollback
Canary	Send 1-5% of traffic to the new version	Catch problems on few users
Feature flags	Ship code dark, enable per user	Decouple deploy from release

Key ideas

- Fast, reliable pipelines let teams ship small changes, which are easier to debug and roll back.
- Run unit tests, integration tests, linting, security scans and (for ML) data and model validation in the pipeline.
- Infrastructure as code (Terraform) and GitOps keep environments reproducible.
- Always have a tested **rollback** path; database migrations should be backward compatible (expand, migrate, contract).
- Tools: GitHub Actions, GitLab CI, Jenkins, Argo CD.

INTERVIEW TIP
Mention canary releases with automatic rollback on error-rate spikes. It combines delivery with observability.

PART 8

Interview and Classic Designs

Putting all the building blocks to work.

- How to Crack a System Design Interview
- Design a URL Shortener
- Design a News Feed
- Design YouTube

CHAPTER 44

How to Crack a System Design Interview

A repeatable framework beats improvisation.

System design interviews are open-ended on purpose. The interviewer is not hunting for one right answer. They want to see how you clarify, structure, prioritise and defend trade-offs. A steady framework keeps you calm and in control for the 45 minutes.

Step	Time	What to do
1. Clarify requirements	5 min	Functional (what it does), non-functional (scale, latency, availability, consistency). Ask about users and scope
2. Estimate	5 min	Users, requests per second, storage, bandwidth. Round numbers, state assumptions
3. API and data model	5 min	Core endpoints and main entities
4. High-level design	10 min	Boxes and arrows: client, LB, services, DB, cache, queue, CDN
5. Deep dive	15 min	Pick the hardest components (bottlenecks) and go deep
6. Wrap up	5 min	SPOFs, monitoring, future scaling, trade-offs you made

Key ideas

- **Think aloud.** Silence hides good thinking. Narrate options and why you choose one.
- Start simple (one server and a database) and evolve it as numbers demand. Do not open with Kafka.
- Always state trade-offs: 'I chose X over Y because Z, at the cost of W.'
- Treat the interviewer as a teammate. Check in: 'Should I go deeper on storage or the feed?'
- Common pitfalls: skipping requirements, over-engineering, ignoring failure modes, never doing the maths.
- Practise with a timer, on a whiteboard or blank page, and review your own recordings.

INTERVIEW TIP

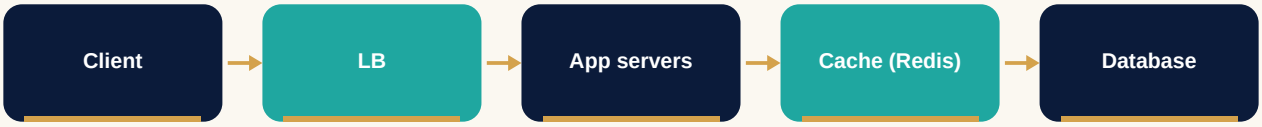
If you only remember one thing: clarify, estimate, design simply, scale where the numbers hurt, and name the trade-offs.

CHAPTER 45

Design a URL Shortener

Think bit.ly: turn a long link into a short one.

Requirements. Create a short URL from a long one, redirect fast, optional custom aliases and expiry, basic click analytics. Highly available and low latency for redirects. **Estimate.** 100 million new URLs a month is about 40 writes per second, and a 100:1 read ratio gives about 4,000 redirects per second. Five years is roughly 6 billion URLs, at about 500 bytes each, around 3 TB.



Key ideas

- **API:** POST /shorten {longUrl, alias?, ttl?} returns the short code; GET /{code} replies **301/302** redirect to the long URL.
- **Short code length:** Base62 (a-z, A-Z, 0-9) with 7 characters gives 62 to the power of 7, about 3.5 trillion combinations.
- **Generating codes:** (a) hash the URL and take 7 characters, handling collisions; (b) a counter converted to Base62; (c) a pre-generated key service handing out unused keys. A distributed counter with ranges per server avoids a bottleneck and collisions.
- **Storage:** key-value lookup (code to long URL) fits a NoSQL store or a sharded SQL table by code. Reads dominate, so cache hot codes (LRU) in Redis.
- **301 vs 302:** 301 is cached by browsers (less load, no analytics); 302 hits you every time (better analytics, easy changes).
- **Analytics:** emit click events to a queue and aggregate asynchronously, so redirects stay fast.
- **Extras:** rate limit creation, block malicious URLs, expire old entries, replicate across regions, use a CDN or edge function for redirects.

INTERVIEW TIP

The interesting question is unique-ID generation without collisions at scale. Compare hashing, counters and a key service out loud.

CHAPTER 46

Design a News Feed

Think Facebook or LinkedIn home timeline.

Requirements. Users follow others, post content and see a ranked, personalised feed of friends' posts. Feed loads in under 200 ms. Posts may appear a few seconds late (eventual consistency is fine).
Estimate. With 500 million daily users opening the feed 10 times each, that is about 58,000 feed reads per second, far more than writes.



Approach	How	Pros	Cons
Fan-out on write (push)	On post, copy the ID into each follower's feed list	Very fast reads	Slow, heavy for celebrities with millions of followers
Fan-out on read (pull)	On open, gather recent posts of everyone you follow	Cheap writes, no wasted work	Slow reads
Hybrid	Push for normal users, pull for celebrities, merge at read	Balanced	More complex

Key ideas

- **Components:** Post service, Social graph service (who follows whom), Fan-out workers fed by a queue, Feed cache (Redis sorted sets of post IDs per user), Post store, Media on object storage plus CDN.
- Store only post **IDs** in feed caches; hydrate the content from a post cache when serving.
- **Ranking:** start with reverse-chronological, then add ML ranking using recency, affinity, engagement and content type.
- **Pagination:** cursor-based, so new posts do not shift pages.
- **Inactive users:** do not precompute feeds for people who have not logged in for weeks; build on demand.

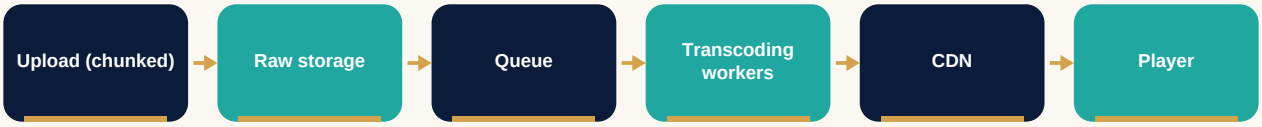
REAL WORLD
The celebrity problem (one post, 100 million followers) is the classic deep dive. The answer is hybrid fan-out.

CHAPTER 47

Design YouTube

Think video upload, processing and global streaming.

Requirements. Users upload videos, others stream them smoothly on any device and network, with search, comments and recommendations. Reliable uploads, low start-up delay, no buffering. **Estimate.** Storage dominates. If 500 hours of video are uploaded every minute and each hour yields several GB across formats, that is petabytes per day, so object storage and tiering are essential.



Key ideas

- **Upload:** resumable, chunked uploads straight to object storage (S3, GCS) via pre-signed URLs, not through app servers.
- **Processing pipeline:** a queue triggers workers that split the video into segments, transcode to many resolutions and bitrates (240p to 4K) and codecs (H.264, VP9, AV1), generate thumbnails and extract metadata. This is a DAG of parallel tasks.
- **Streaming:** adaptive bitrate (HLS or MPEG-DASH): the video is cut into 2-10 second segments at several qualities and the player picks the best one for current bandwidth.
- **Delivery:** CDN is the heart of the system. Popular videos sit at the edge; the long tail is served from regional or origin storage.
- **Metadata:** sharded SQL or NoSQL for titles, owners and counts; Elasticsearch for search; view counts via a stream (Kafka) and batched updates (eventual consistency).
- **Recommendations:** offline ML pipelines build candidates; an online ranker orders them per user.
- **Also consider:** copyright and content moderation, DRM, cold storage for rarely watched videos, and cost, since bandwidth and storage are the biggest bills.

INTERVIEW TIP

Two sentences to nail it: 'Upload goes direct to object storage, a queue drives parallel transcoding into adaptive segments, and a CDN serves them.'

PART 9
Advanced Topics

Where the foundation turns into engineering depth.

- Back-of-the-Envelope Estimation
- CAP and PACELC
- Consensus: Raft and Paxos
- Leader Election and Distributed Locks
- Distributed Transactions: 2PC and Saga
- CQRS and Event Sourcing
- Bloom Filters and Probabilistic Structures
- B-Trees vs LSM Trees
- Distributed ID Generation
- Microservices, API Gateway and Service Mesh
- Kafka and Stream Processing
- Search Systems
- Geospatial Indexing
- Backpressure and Load Shedding
- Gossip, Clocks and Ordering
- Security Essentials
- Zero-Downtime Deployments
- System Design for AI and ML

CHAPTER 48

Back-of-the-Envelope Estimation

Fast maths that guides every design decision.

Estimation tells you whether you need one server or one thousand, one disk or one data centre. Use round numbers, powers of two and ten, and write down assumptions.

Quantity	Rule of thumb
Seconds in a day	about 100,000 (86,400)
1 million requests per day	about 12 requests per second
1 billion requests per day	about 12,000 requests per second
Peak traffic	2 to 5 times the average
1 KB, 1 MB, 1 GB, 1 TB	10^3, 10^6, 10^9, 10^12 bytes
One server handles	roughly 1,000 to 10,000 simple req/s
Text post / image / video minute	about 1 KB / 300 KB / 50 MB

Key ideas

- **Worked example:** 10 million daily users, 20 requests each = 200 million requests per day, about 2,300 per second average, about 7,000 at peak.
- Storage: 1 million new photos a day at 500 KB is 500 GB a day, about 180 TB a year (before replication).
- Bandwidth = requests per second times average response size.

> **INTERVIEW TIP**
> Accuracy does not matter, order of magnitude does. Say 'about' and move on.

CHAPTER 49

CAP and PACELC

The fundamental limit of distributed data.

The **CAP theorem** says that when a network **P**artition happens, a distributed store must choose between **C**onsistency (every read sees the latest write) and **A**vailability (every request gets an answer). Partitions are a fact of life, so the real choice is CP or AP.

Type	Behaviour during partition	Examples
CP	Refuses some requests to stay correct	ZooKeeper, etcd, HBase, MongoDB (default settings), Spanner
AP	Keeps answering, may return stale data	Cassandra, DynamoDB, Riak, CouchDB

Key ideas

- **PACELC** extends it: if there is a Partition choose A or C; **E**lse (normal operation) choose **L**atency or **C**onsistency. Even without failures, stronger consistency costs latency.

- Many systems are tunable. Cassandra and DynamoDB let you choose quorum sizes per query (R + W greater than N gives strong reads).
- Choose per feature: payments want CP, likes and view counters want AP.

> **REMEMBER**
> CAP is about behaviour during partitions only. It does not mean you must always give up one letter.

CHAPTER 50

Consensus: Raft and Paxos

Getting machines to agree despite failures.

Consensus lets a cluster agree on a value or a sequence of operations even when nodes crash or messages are delayed. It underpins leader election, configuration stores, locks and replicated databases.



Key ideas

- **Raft** (designed to be understandable) splits the problem into leader election, log replication and safety. Used by etcd, Consul, CockroachDB and TiKV.
- **Paxos** is the original, famously subtle, protocol. Multi-Paxos and variants run inside Google Spanner and Chubby. **ZAB** powers ZooKeeper.
- Needs a **majority quorum**: a 3-node cluster tolerates 1 failure, 5 nodes tolerate 2. Use odd sizes.
- Consensus is slow and does not scale writes. Use it for small, critical metadata, not bulk data.
- Byzantine fault tolerance (PBFT, blockchains) handles malicious nodes and is a different, costlier problem.

> **INTERVIEW TIP**
> You rarely implement consensus. You use etcd or ZooKeeper. Know when you need it: leader election, locks, and configuration.

CHAPTER 51

Leader Election and Distributed Locks

Making sure only one node does the job.

Some work must be done by exactly one node at a time: running a scheduled job, being the primary database, owning a partition. **Leader election** chooses that node and replaces it on failure. A **distributed lock** gives temporary exclusive access to a resource.

Key ideas

- Implement with a consensus store (etcd leases, ZooKeeper ephemeral nodes) or the database's own mechanisms. Avoid home-grown election.
- Always use a **lease with an expiry** and heartbeat renewal so a dead leader releases leadership.
- **Fencing tokens**: each lock grant gets an increasing number; the resource rejects writes with an older token. This defends against a paused 'zombie' leader that wakes up after its lease expired.
- Redis-based locks (Redlock) are debated for correctness-critical uses. For safety use a consensus system or database constraints.
- Prefer designs that need no lock: idempotent operations, partitioned ownership, optimistic concurrency with version numbers.

TRADE-OFF
Locks across machines are slow and fragile. Every time you reach for one, ask if partitioning or idempotency can remove the need.

CHAPTER 52

Distributed Transactions: 2PC and Saga

One business action across many services.

When an order touches payment, inventory and shipping in separate databases, a single ACID transaction is not available. Two main patterns exist.



Pattern	How it works	Pros	Cons
Two-Phase Commit (2PC)	Coordinator asks all to Prepare, then tells all to Commit or Abort	Strong atomicity	Blocking, slow, coordinator is a SPOF
Saga	Chain of local transactions; if one fails, run compensating actions in reverse	Scalable, no global locks	Eventual consistency, compensation logic is hard

Key ideas

- Saga **choreography**: services react to each other's events. **Orchestration**: a central saga coordinator drives the steps (easier to follow and monitor).
- Compensations must be idempotent and always possible (refund, cancel, restock). Some actions cannot be undone (an email sent), so order steps with the irreversible ones last.

- **Transactional outbox**: write the event to an outbox table in the same local transaction as the data, then publish it reliably. This avoids losing events or publishing events for rolled-back data.
- Tools: Temporal, Camunda, AWS Step Functions.

INTERVIEW TIP
Say 'Saga with an outbox and idempotent steps' for microservice workflows. Mention 2PC only to explain why you avoid it.

CHAPTER 53

CQRS and Event Sourcing

Separate writing from reading, and store history as the truth.

CQRS (Command Query Responsibility Segregation) uses different models for changing data (commands) and reading it (queries). The write side enforces rules; the read side holds denormalised views tuned for fast queries, updated through events. **Event sourcing** stores every change as an immutable event; current state is derived by replaying them.



Key ideas

- Why: read and write loads and shapes differ greatly, so scaling and optimising them separately pays off.
- Event sourcing gives a perfect audit trail, time travel, and the ability to build new read views from history. Banking ledgers are the natural example.
- Costs: eventual consistency between sides, event schema evolution, steeper learning curve, snapshots needed for long histories.
- Use selectively for complex, high-value domains. A plain CRUD app does not need either.

TRADE-OFF
CQRS and event sourcing are powerful and easy to over-apply. Reach for them when the pain is real, not for fashion.

CHAPTER 54

Bloom Filters and Probabilistic Structures

Tiny memory, clever answers, small chance of being wrong.

A **Bloom filter** answers 'is X in this huge set?' using a small bit array and several hash functions. It can say **'definitely not'** or **'probably yes'**: false positives are possible, false negatives never.

Structure	Answers	Used for
Bloom filter	Is item in the set?	Skip disk lookups (Cassandra, LSM stores), block known bad URLs, cache penetration guard
Count-Min Sketch	Roughly how often did X occur?	Top-K, trending, heavy hitters
HyperLogLog	How many distinct items?	Unique visitors, Redis PFCOUNT, using about 12 KB for billions
Cuckoo filter	Like Bloom, supports deletion	Dynamic sets

Key ideas

- Typical Bloom filter: about 10 bits per item gives roughly a 1% false-positive rate.
- Perfect when the cheap 'definitely not' answer saves an expensive lookup most of the time.
- Standard Bloom filters cannot delete items. Use counting or cuckoo variants if you need that.

REAL WORLD
Before reading from disk, an LSM database asks a Bloom filter 'might this key be in this file?' and skips most files entirely.

CHAPTER 55

B-Trees vs LSM Trees

How databases store data on disk.

Storage engines make a core trade-off between fast reads and fast writes.

	B-Tree	LSM Tree
Write path	Update pages in place (random I/O)	Append to memtable and log, flush sorted files (sequential I/O)
Read path	Fast, one tree walk	May check several files; helped by Bloom filters
Maintenance	Page splits	Background compaction merges files
Strength	Read-heavy, predictable latency	Write-heavy, high ingest
Used by	PostgreSQL, MySQL (InnoDB), most SQL	Cassandra, RocksDB, LevelDB, HBase

Key ideas

- **Write amplification**: LSM trees rewrite data during compaction; B-trees rewrite pages. Both multiply the physical writes behind one logical write.
- **Write-ahead log**: both append changes to a sequential log first so crashes can be recovered.

- Choose the engine to match the workload, not the other way round.

> **INTERVIEW TIP**
> If the design is write-heavy (events, metrics, messages), mention LSM-based stores such as Cassandra. If transactional, a B-tree SQL database.

CHAPTER 56

Distributed ID Generation

Unique IDs without a central bottleneck.

Auto-increment IDs break when data is sharded across machines. You need IDs that are unique, ideally sortable by time, and generated without coordination.

Option	Pros	Cons
UUID v4	No coordination, simple	128 bits, random so poor index locality
UUID v7 / ULID	Time-ordered, good for indexes	Larger than 64 bits
DB ticket server	Simple, sequential	Central bottleneck and SPOF
Snowflake ID	64-bit, time-sortable, no coordination	Needs unique worker IDs and sane clocks

Key ideas

- **Snowflake** layout (Twitter): 1 unused bit, 41 bits of millisecond timestamp, 10 bits machine ID, 12 bits sequence. That is about 4,096 IDs per millisecond per machine and roughly 69 years of range.
- Time-sortable IDs give you free chronological ordering and efficient range queries and pagination.
- Clock skew is the danger: a clock that moves backwards can produce duplicates, so the generator must refuse or wait.

> **REAL WORLD**
> Instagram, Discord and many others use Snowflake-style IDs so that sorting by ID equals sorting by time.

CHAPTER 57

Microservices, API Gateway and Service Mesh

Splitting a system into independently deployable pieces.

A **monolith** is one deployable unit; **microservices** split the system by business capability, each owning its data and deployed independently.

	Monolith	Microservices
Simplicity	High, one codebase	Low, many moving parts
Scaling	Scale everything together	Scale each service on its own
Team autonomy	Limited	High
Failure modes	In-process calls	Network calls, partial failures, latency
Best for	Small teams, early products	Large orgs with clear domain boundaries

Key ideas

- **API Gateway**: single entry point handling routing, authentication, rate limiting, TLS, request aggregation and caching (Kong, AWS API Gateway, Envoy).
- **Service mesh** (Istio, Linkerd): sidecar proxies manage service-to-service traffic, giving mTLS, retries, circuit breaking and telemetry without changing application code.
- Each service owns its database. Sharing a database makes a 'distributed monolith', the worst of both worlds.
- Start with a well-structured **modular monolith** and split out services when team size or scaling needs demand it.

TRADE-OFF
Microservices trade code complexity for operational complexity. Do not adopt them to look modern.

CHAPTER 58

Kafka and Stream Processing

A durable, replayable log at the centre of the data platform.

Apache Kafka stores events in partitioned, replicated, append-only **logs**. Producers write; consumer groups read at their own pace and can rewind. Order is guaranteed *within* a partition, so choose the partition key (such as user ID) carefully.



Key ideas

- Throughput: millions of messages per second on modest clusters thanks to sequential disk writes and batching.
- Consumer **offsets** track progress. Adding consumers to a group (up to the partition count) scales processing.
- **Stream processing** (Flink, Kafka Streams, Spark Streaming) does windowed counts, joins and detection in real time.
- **Lambda architecture** runs batch and stream paths side by side; **Kappa** uses a single streaming path with replay.
- **CDC (Change Data Capture)** (Debezium) streams database changes into Kafka to keep caches, search and analytics in sync.
- Exactly-once semantics exist within Kafka via idempotent producers and transactions, but end-to-end still needs idempotent sinks.

INTERVIEW TIP
Use Kafka when you need replay, multiple independent consumers, or very high throughput. For simple job queues, SQS or RabbitMQ is enough.

CHAPTER 59

Search Systems

Finding words in billions of documents in milliseconds.

Databases are poor at free-text search. Search engines (Elasticsearch, OpenSearch, Solr) build an **inverted index**: for each word, a list of the documents that contain it. A query looks up each term and intersects the lists.



Key ideas

- Text analysis: lowercase, remove stop words, stemming (running to run), synonyms, n-grams for autocomplete.
- **Relevance** uses TF-IDF or BM25, plus signals such as freshness, popularity and personalisation.
- Index is sharded and replicated; a query fans out to all shards and merges results (scatter-gather).
- Keep the search index as a **derived copy** of your source-of-truth database, updated through events or CDC.
- Autocomplete: prefix trie or edge n-grams. **Semantic search** uses embeddings and vector indexes (HNSW) to match meaning, a natural bridge from this repo's AI modules.

> **REAL WORLD**
> Modern search often blends keyword (BM25) and vector search, then re-ranks with a model. That is the retrieval half of RAG.

CHAPTER 60

Geospatial Indexing

Finding 'what is near me' quickly.

Searching nearby drivers, restaurants or friends by comparing latitude and longitude against every row is far too slow. Spatial indexes organise the map so nearby points are found without a full scan.

Technique	Idea	Used by
Geohash	Encode a location as a string; shared prefix means close	Redis GEO, many databases
Quadtree	Recursively divide map into four, split where dense	Custom proximity services
S2 cells	Map the sphere onto a hierarchy of cells	Google Maps, Uber (H3 is hexagonal)
R-tree	Bounding rectangles in a tree	PostGIS, spatial databases

Key ideas

- Query pattern: find the cell of the user, search that cell and neighbouring cells, then sort by exact distance.
- Moving objects (drivers) produce constant updates. Keep them in memory (Redis) with a short TTL rather than a disk database.

- Edge case: points near a cell border, so always check the adjacent cells.

> **INTERVIEW TIP**
> For Uber, Yelp or Tinder style problems, name geohash or quadtree and explain neighbour cell lookup.

CHAPTER 61

Backpressure and Load Shedding

What to do when demand exceeds capacity.

Every system has a limit. Without protection, overload turns into a spiral: queues grow, latency rises, clients retry, load rises further and the system collapses. **Backpressure** pushes the slowdown upstream; **load shedding** deliberately drops low-priority work to stay alive.

Key ideas

- Use **bounded queues**. An unbounded queue is just a delayed outage and a memory leak.
- Reject early with 429 or 503 and a Retry-After instead of timing out slowly.
- **Prioritise** traffic: protect logins and payments, shed analytics and recommendations first.
- **Graceful degradation**: serve cached or simplified results, disable expensive features, show a static page.
- Concurrency limits and adaptive throttling (such as latency-based limits) keep servers inside their efficient range.
- Combine with rate limiting, circuit breakers and capacity planning.

> **REAL WORLD**
> During a big sale or incident, successful platforms switch off recommendations and keep checkout alive. Degrade features, not the core.

CHAPTER 62

Gossip, Clocks and Ordering

Time is slippery in distributed systems.

Machines have different clocks that drift, so 'which event came first?' is hard. Wall-clock timestamps can mislead, and ordering needs other tools.

Concept	Idea
Lamport clock	A counter that gives a consistent logical order of causally related events
Vector clock	A counter per node; detects whether two updates were concurrent (a conflict)
Hybrid logical clock	Physical time plus logical counter; used by CockroachDB
TrueTime (Spanner)	GPS and atomic clocks give a bounded uncertainty; the DB waits it out for global consistency
Gossip protocol	Nodes periodically share state with random peers so information spreads like a rumour

Key ideas

- Gossip powers membership and failure detection in Cassandra, Consul and Dynamo-style systems. It is simple, scalable and robust, but only eventually consistent.
- Never rely on exact timestamps from different machines to decide order. Use sequence numbers, logical clocks or a consensus log.
- Last-write-wins silently drops data when clocks disagree.

REMEMBER
In distributed systems, 'now' does not exist globally. Order events by causality, not by wall clocks.

CHAPTER 63

Security Essentials

Design security in, not on top.

Security belongs in the first design sketch. A concise checklist covers most interview and real-world needs.

Area	Practice
Authentication	Who are you: passwords with bcrypt or argon2, MFA, passkeys, OAuth 2.0 and OpenID Connect for login with Google or GitHub
Authorisation	What may you do: RBAC or ABAC, least privilege, check on the server for every request
Transport	TLS everywhere, HSTS; mTLS between internal services
Data	Encrypt at rest, manage keys in a KMS or vault, never store secrets in code
Input	Validate and parameterise queries (SQL injection), escape output (XSS), CSRF tokens
Abuse	Rate limiting, WAF, DDoS protection, bot detection
Audit	Immutable audit logs, alert on suspicious behaviour

Key ideas

- **JWTs** are signed, not encrypted. Do not put secrets in them, keep them short-lived and verify signature, expiry and audience.
- **Zero trust**: never trust a request just because it comes from inside the network.
- Plan for privacy and regulation (GDPR, data residency, right to delete) in your data design.
- For AI systems add prompt injection defence, data-leak controls and model access limits.

INTERVIEW TIP
Add one security slide at the end of any design: authN, authZ, TLS, encryption at rest and rate limiting.

CHAPTER 64

Zero-Downtime Deployments

Changing the engine while the plane is flying.

Users should never notice a release. This takes more than a deployment strategy; the whole system, including the database, must be designed for versions to coexist.

Key ideas

- **Rolling, blue-green and canary** releases keep old and new code running side by side during the switch.
- **Backward-compatible changes**: new code must work with the old schema and old code with the new schema. Use **expand, migrate, contract**: add the new column, backfill and dual-write, switch reads, then remove the old column in a later release.
- **Graceful shutdown**: stop accepting new connections, finish in-flight requests, then exit. Honour readiness probes.
- **Feature flags** separate deploying code from releasing a feature and give an instant kill switch.
- Version APIs and message schemas (use a schema registry with compatibility rules).
- Automate rollback on error-rate or latency regression.

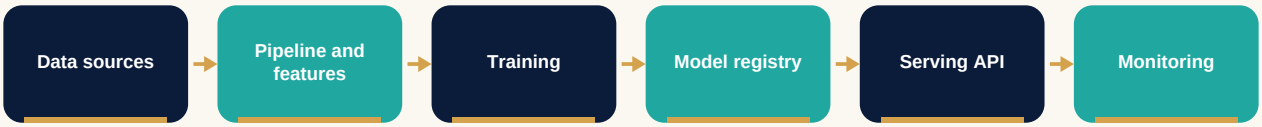
> **REAL WORLD**
> The riskiest part of most releases is not the code but the database migration. Make migrations small, reversible and separate from code deploys.

CHAPTER 65

System Design for AI and ML

Where this module meets the rest of AI Zero to Hero.

ML systems add new pieces to everything above: data pipelines, training, model registries and serving. The same principles of scalability, reliability and observability apply, with some twists.



Key ideas

- **Serving:** batch inference (nightly, cheap) vs online inference (low latency). Use model servers (TorchServe, Triton, vLLM), batching and GPU autoscaling.
- **LLM apps:** cache prompts and responses, stream tokens over SSE or WebSockets, rate limit per user and per token, queue long jobs, and add fallbacks to a smaller model.
- **RAG architecture:** ingest documents, chunk, embed, store in a vector database, retrieve, re-rank, then generate. The index is a derived store kept fresh by events.
- **Feature stores** keep training and serving features consistent and avoid training-serving skew.
- **Monitoring:** data drift, prediction quality, latency, cost per request and GPU utilisation, in addition to normal service metrics.

- **Cost** is a first-class constraint: GPU time is expensive, so batching, quantisation and caching matter.

INTERVIEW TIP
For ML design questions, cover data, training, serving, monitoring and feedback loop, then apply the usual scale and reliability tools to each.

PART 10

Project Learning

Three famous products, simplified. Draw each one yourself before reading.

- Instagram Architecture
- Twitter (X) Architecture
- WhatsApp Architecture
- Compare and Learn

A note on accuracy

REMEMBER
These case studies are **simplified, educational reconstructions** based on engineering talks and public write-ups. Real internals change constantly and are not fully public. Use them to learn patterns, not as exact blueprints. Each study follows the same steps: requirements, scale, architecture, deep dive, lessons.

CHAPTER 66

Instagram Architecture

A photo and video sharing network built on caching, sharding and a simple stack.

Core features: upload photos and videos, follow users, home feed, Stories, Reels, likes and comments, search and explore, notifications, direct messages. **Scale:** billions of users, hundreds of millions of uploads per day, extremely read-heavy, with media dominating storage and bandwidth.



Key ideas

- **Upload path:** the app uploads media (resumable) to object storage. A queue (Celery or similar) triggers processing: resize into multiple resolutions, compress, apply filters, extract metadata, run moderation. The CDN then serves the variants.
- **Data:** users, follows and metadata live in sharded PostgreSQL (early Instagram famously scaled Postgres with logical sharding and custom IDs); high-volume activity data suits Cassandra-like stores. Social graph reads are served by a heavy caching layer (Meta's TAO).
- **IDs:** Instagram used a Snowflake-style 64-bit ID built from time, shard ID and sequence generated inside PostgreSQL, so IDs are sortable and encode the shard.
- **Feed:** a hybrid of precomputed candidates and ranking. Early versions were chronological; today a machine-learning ranker scores posts using relationship, interest and recency signals.
- **Stories:** expire after 24 hours, which makes TTL-based storage and caching a natural fit.
- **Explore and Reels:** offline pipelines build candidate sets and embeddings; online ranking personalises results within tens of milliseconds.
- **Reliability:** multi-region replication, heavy monitoring, feature flags and canary rollouts.

REAL WORLD
Lesson: Instagram grew to millions of users with a tiny team by choosing simple, proven tools (Django, PostgreSQL, Redis, Memcached) and aggressive caching, scaling out only where metrics demanded.

Instagram: interview deep-dive questions

- How would you generate sortable, shard-aware photo IDs?
- How do you serve the same photo in several sizes to many device types cheaply?
- How do you show an accurate like count for a post with 50 million likes? (Eventual counters, sharded counters, approximate display.)
- How would you design Stories expiry and the 'who viewed' list?

CHAPTER 67

Twitter (X) Architecture

A real-time, read-heavy firehose of short messages.

Core features: post short messages, follow users, home timeline, search, trends, notifications, direct messages, media. **Scale:** hundreds of millions of users; timeline reads outnumber writes by orders of magnitude; fan-out is the central challenge, especially for accounts with tens of millions of followers.



Key ideas

- **Tweet storage:** tweets are stored by ID in a distributed key-value store (Twitter built Manhattan; earlier versions used sharded MySQL). IDs come from **Snowflake**, which Twitter created.
- **Home timeline:** classic fan-out on write. When you tweet, a fan-out service inserts the tweet ID into the in-memory timeline list (Redis) of each follower, so reading a timeline is one cache read.
- **Celebrity problem:** for accounts with huge follower counts, writing to every list is too slow and wasteful. Their tweets are **pulled at read time** and merged with the precomputed timeline (hybrid).
- **Social graph:** a dedicated graph service stores follow relationships (Twitter's FlockDB) and answers 'who follows whom' quickly.
- **Search:** a real-time inverted index (Earlybird, built on Lucene) keeps recent tweets searchable within seconds; older tweets live in a larger, slower tier.
- **Trends:** a streaming pipeline counts terms and hashtags in sliding windows, detects sudden spikes against baselines and filters spam (Count-Min Sketch style structures help).
- **Media:** images and video go to object storage and a CDN; videos are transcoded for adaptive streaming.
- **Notifications and real-time:** a push infrastructure plus streaming connections for live updates; Kafka-style logs connect services.
- **Ranking:** 'For You' uses candidate retrieval from the network and beyond, then a heavy ranking model, then filters and mixing of ads.

> **INTERVIEW TIP**
> Twitter is the textbook example of fan-out on write versus fan-out on read and the hybrid compromise. Be ready to draw it and quantify why.

Twitter: interview deep-dive questions

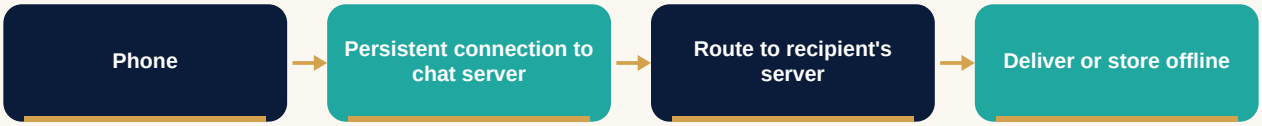
- What happens, step by step, when a user with 50 million followers tweets?
- How do you keep recent-tweet search fresh within seconds?
- How do you detect a trending topic without counting every hashtag exactly?
- How do you handle a viral tweet (hot key) hammering one cache node?

CHAPTER 68

WhatsApp Architecture

Billions of private messages with a famously small engineering team.

Core features: one-to-one and group chat, delivery and read receipts, online status, media sharing, voice and video calls, end-to-end encryption, multi-device. **Scale:** billions of users and tens of billions of messages per day. Latency and reliability matter more than anything; messages must never be lost.



Key ideas

- **Connections:** each client holds a persistent connection (a custom protocol derived from XMPP over TCP with TLS-style encryption) to a chat gateway. Servers are built on **Erlang**, whose lightweight processes handle millions of concurrent connections per cluster of machines.
- **Send path:** sender to its server, server finds the recipient's connection through a session or presence registry and forwards it. The server returns the single, double and blue-tick acknowledgements as the message is accepted, delivered and read.
- **Store and forward:** if the recipient is offline, the encrypted message waits in a queue or temporary store until the device reconnects, then is delivered and deleted from the server. Servers are a relay, not a permanent archive.
- **End-to-end encryption:** uses the Signal protocol (X3DH key agreement and the Double Ratchet). Servers see only ciphertext and metadata. Group chats use sender keys so one encryption fans out efficiently.
- **Groups:** a message to a group is fanned out per member by the server; membership data is stored centrally and cached.
- **Media:** the client encrypts media, uploads the blob to storage and sends the recipient a small message with the link and decryption key. Media is stored temporarily and fetched through a CDN.
- **Multi-device:** each device has its own keys; messages are encrypted to every linked device.
- **Presence and typing indicators:** small ephemeral updates, sent best-effort and never stored.
- **Reliability:** per-conversation sequencing, client-side retries with message IDs for deduplication (idempotency), and regionally distributed gateways with fast failover.

REAL WORLD
Lesson: pick a technology that fits the problem. Erlang's concurrency model let a very small team run a global messaging service, and focusing on a simple store-and-forward design kept the backend lean.

WhatsApp: interview deep-dive questions

- How do you route a message to the right chat server when millions of users are connected?
- How do you guarantee ordering and no-loss delivery, even when the sender retries?
- How would you design group chat for 1,000 members and the read-receipt storm it creates?
- How do you support multi-device end-to-end encryption?

CHAPTER 69

Compare and Learn

The same building blocks, assembled differently.

Notice how three very different products reuse the ideas from this module, chosen according to their dominant requirement.

	Instagram	Twitter (X)	WhatsApp
Main challenge	Media storage and delivery	Fan-out and real-time search	Persistent connections and delivery
Traffic shape	Read-heavy, large media	Read-heavy, small bursts	Balanced, latency-critical
Key patterns	CDN, sharding, caching, ranking	Fan-out hybrid, Snowflake, inverted index	WebSockets-like persistent links, store and forward, E2EE
Consistency	Eventual for feeds, counts	Eventual for timelines	Strict per-chat ordering, at-least-once with dedup
Data stores	PostgreSQL, Cassandra, cache layer	Key-value store, Redis, search index	Custom in-memory and temporary stores
Async work	Media processing, notifications	Fan-out, trends, indexing	Offline queue, push notifications

Key ideas

- Every design starts from requirements and numbers. The architecture is a consequence of them.
- Caching, queues, sharding, replication, CDNs and observability appear in all three.
- Practise: pick a fourth product (Telegram, TikTok, Uber, Netflix) and design it using this framework.

WHAT NEXT?

Keep building.

System design is learned by drawing, estimating and explaining, over and over.

- Redraw each chapter diagram from memory.
- Re-design the URL shortener, news feed and YouTube with a 45-minute timer.
- Walk through Instagram, Twitter and WhatsApp aloud, naming trade-offs.
- Follow the next modules in AI Zero to Hero for deeper advanced topics.
- Build a small project: a rate-limited API with Redis cache, a queue and a worker.

Sheharyar Sarmad

LinkedIn: linkedin.com/in/sheharyar-sarmad-9b7736289

GitHub: github.com/Sheharyar-Sarmad/ai-zero-to-hero

This module: 25_systemdesign_foundation

Companion resource: freesystemdesign.com

If this guide helped you, star the repo and connect with me on LinkedIn.